

Parent ↔ Child Communication

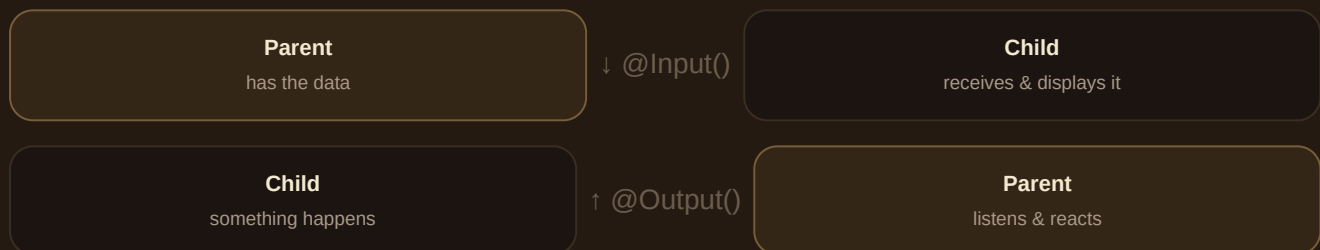
How two components talk to each other — a Parent **gives** data down to a Child with `@Input()`, and a Child **tells** the Parent something happened with `@Output()`.

1 · THE BIG PICTURE

What is component communication?

A real app is built from many small components nested inside each other — a Parent containing one or more Children. Sometimes the Parent needs to **give** data to the Child (e.g. a product to display). Sometimes the Child needs to **tell** the Parent something happened (e.g. "the user clicked Add to Cart").

Real-world analogy: a Parent gives pocket money to their Child (Parent → Child). The Child later tells the Parent "I spent it!" (Child → Parent).



2 · DIRECTION 1

@Input() — Parent sends data to the Child

Child declares the input

```
// child.component.ts
import { Component, Input } from
 '@angular/core';

@Component({ selector: 'app-child' })
export class ChildComponent {
  @Input() message: string = '';
}
```

Parent passes it down

```
<!-- parent.component.html -->
<app-child [message]="parentMessage"></app-
child>

// parent.component.ts
export class ParentComponent {
  parentMessage = 'Hello from Parent!';
}
```

The square brackets `[message]` are **Property Binding** — they connect the Parent's value to the Child's `@Input()` property automatically. Data flows in **one direction only**: Parent → Child. The Child should not change an `@Input()` value directly — think of it as a labeled slot: "Parent, you can plug a value in here."

@Output() — Child sends an event to the Parent

Child declares the output

```
// child.component.ts
@Output() notifyParent = new
EventEmitter<string>();

sendMessage() {
  this.notifyParent.emit('Button clicked!');
}

<!-- child.component.html -->
<button (click)="sendMessage()">Click
me</button>
```

Parent listens for it

```
<!-- parent.component.html -->
<app-child
(notifyParent)="onChildMessage($event)">
</app-child>

// parent.component.ts
onChildMessage(msg: string) {
  console.log('Child said:', msg);
}
```

The round brackets (`notifyParent`) are **Event Binding** — the exact same syntax as (`click`). `$event` carries whatever value the Child passed to `.emit(...)`.

4 · A FULL REAL-WORLD EXAMPLE

A product card that reports back to its parent

```
// product-card.component.ts
@Component({ selector: 'app-product-card' })
export class ProductCardComponent {
  @Input() product!: Product;
  @Output() addToCart = new EventEmitter<Product>();

  onAddClick() {
    this.addToCart.emit(this.product);
  }
}

<!-- product-card.component.html -->
<h3>{{ product.name }}</h3>
<button (click)="onAddClick()">Add to Cart</button>

<!-- product-list.component.html (Parent) -->
<app-product-card
  *ngFor="let p of products"
  [product]="p"
  (addToCart)="onProductAdded($event)">
</app-product-card>

// product-list.component.ts
onProductAdded(product: Product) {
  this.cartService.add(product);
}
```

One `ProductCardComponent` is reused for every product in the list — each instance receives its own `product` via `@Input()`, and reports back to the *same* parent handler via `@Output()` when its own "Add to Cart" button is clicked.

5 · SIDE BY SIDE

@Input() vs. @Output()

@INPUT() — PARENT → CHILD	@OUTPUT() — CHILD → PARENT
Parent passes data down using [property]	Child announces an event using EventEmitter
Child receives it as a normal property	Parent listens using (eventName)
Used to configure / feed data into a Child	Used to notify the Parent something happened
Example: passing a product object to a card	Example: telling the Parent "Add to Cart" was clicked

6 · COMMON MISTAKES

What trips people up

- Mutating an `@Input()` value inside the Child — treat it as read-only; if the Child needs its own copy, assign it to a local field first.
- Forgetting the `!` or a default value on a required `@Input()`, causing "possibly undefined" errors.
- Emitting the whole event object instead of the data the Parent actually needs — keep `EventEmitter<T>` typed to something useful.
- Reaching for `@Input()/@Output()` between components that aren't Parent/Child — that's what a shared Service is for.

7 · GOOD TO KNOW

Modern signal-based syntax

Newer Angular versions also offer signal-based alternatives — `input()` and `output()` functions instead of the `@Input()/@Output()` decorators. They work conceptually the same way, just written in a signal-friendly style. `@Input()/@Output()` are still fully supported and appear everywhere in real projects.